

Linear Regression & Binary Classification

Purpose

The purpose of this project is to strengthen your understanding of linear and logistic regression by implementing models. In the first part, you will be implementing the objective function for linear regression, and polynomial regression and use ridge regression to implement the best possible fit. In the second part, you will implement a logistic regression model to predict if a subject is diabetic or not by classifying the subjects using a binary classification model.

Objectives

Learners will be able to

- Understand the core principles of linear regression, including estimating objective function and exploring various techniques to solve linear regression
- Understand logistic neurons, outline the logistic regression objective function, and learn about binary classification and gradient descent to solve logistic regression

Technology Requirements

- GPU environment (optional)
- Jupyter Notebook
- Python3 (Python 3.8 and above)
- Numpy
- Matplotlib

Directions

Accessing ZyLabs

You will complete and submit your work through zyBooks's zyLabs. Follow the directions to correctly access the provided workspace:

1. Go to the course submission page “**Submission: Linear Regression & Binary Classification**”
2. Click the “**Load Submission...in new window**” button.
3. Once in ZyLabs, click the **green button** in the Jupyter Notebook to get started.
4. Review the directions and resources provided in the description.
5. When ready, review the provided code and develop your work where instructed.

Project Directions

Part 1 - Linear Regression

You need to define the objective function that will be optimized by the linear regression model.

$$L(\Phi(X), Y, \theta) = (Y - \Phi(X)\theta)^T (Y - \Phi(X)\theta)$$

Here, $\Phi(X)$ is the design matrix of dimensions $(m(d + 1))$ and Y is the m dimension vector of labels. θ is the $(d+1)$ dimension vector of weight parameters.

Define a closed-form solution to the objective function. Closed-form solution is given by,

$$\theta = (\Phi(X)^T \Phi(X))^{-1} \Phi(X)^T Y$$

Here, $\Phi(X)$ is the $(m(d+1))$ dimension design matrix obtained using the `poly_transformation` function defined earlier, and Y are the ground truth labels of dimensions $(m \times 1)$

Define a function to evaluate the goodness of the linear regression model using root mean square error. This compares the difference between the estimate Y -labels and the ground truth Y -labels. The smaller the RMSE value, the better the fit.

$$\text{RMSE(Root Mean Squared Error)} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_{\text{pred}}^{(i)} - y^{(i)})^2}$$

Here, y_{pred} is the estimated labels of dimensions $(m,1)$ and y is the ground truth labels of dimensions $(m,1)$

Ridge Regression

Similar to Linear Regression, implement the closed-form solution to ridge regression.

The degree of the polynomial regression is $d = 10$. Even though the curve appears to be smooth, it may be fitting to the noise. Hence, you need to use the Ridge Regression to get a smoother fit and avoid overfitting.

Ridge regression objective form:

$$L(\Phi(X), Y, \theta, \lambda) = (Y - \Phi(X)\theta)^T(Y - \Phi(X)\theta) + \lambda^2 \theta^T \theta$$

where, $\lambda \geq 0$ is the regularization parameter. The larger the value λ , the smoother the curve. The closed-form solution to the objective is given by:

$$\theta = (\Phi(X)^T \Phi(X) + \lambda^2 I_d)^{-1} \Phi(X)^T Y$$

Here, I_d is the identity matrix of dimensions $((d+1) \times (d+1))$, $\Phi(X)$ is the $(m(d+1))$ dimension design matrix obtained using `poly_transform` function (from linear regression) and Y are the ground truth labels of dimensions $(m \times 1)$.

Cross Validation to Estimate

To avoid overfitting when using a high-degree polynomial, we have used **ridge regression**. We now need to estimate the optimal value of λ using **cross-validation**.

We will obtain a generic value of λ using the entire training dataset to validate. We will employ the method of **k-fold cross-validation**, where we split the training data into k non-overlapping random subsets. In every cycle, for a given value of λ , $k-1$ subsets are used for training the ridge regression model and the remaining subset is used for evaluating the goodness of the fit. We estimate the average goodness of the fit across all the subsets and select the λ that results in the best fit.

It is easier to shuffle the index and slice the training into the required number of segments than to process the complete dataset.

Refer to the following documentation for splitting and shuffling:

- <https://docs.scipy.org/doc/numpy-1.15.0/reference/generated/numpy.random.shuffle.html>
- <https://docs.scipy.org/doc/numpy/reference/generated/numpy.split.html>

You need to define a function to implement the k-fold cross-validation that takes the following inputs:

- `k_fold`: number of validation subsets
- `train_X`: training data of dimensions $(m, 1)$
- `train_Y`: ground truth training labels
- `lambda`: ridge regularization lambda parameter
- `d`: polynomial degrees

Part 2 Logistic Regression

Machine learning is used in medicine to assist doctors with crucial decision-making based on diagnostic data. In this project, we will be designing a logistic regression model (single-layer neural network) to predict if a subject is diabetic or not. The model will classify the subjects into two groups diabetic (Class 1) or non-diabetic (Class 0) - a binary classification model.

We will be using the 'Pima Indians Diabetes dataset' to train our model which contains different clinical parameters (features) for multiple subjects along with the label (diabetic or not-diabetic). Each subject is represented by 8 features (Pregnancies, Glucose, Blood-Pressure, SkinThickness, Insulin, BMI, Diabetes-Pedigree-Function, Age) and the 'Outcome' which is the class label. The dataset contains the results from 768 subjects.

We will be splitting the dataset into train and test data. We will load and train our model on the train data and predict the categories on the test data.

1. Steps involved in Logistic Regression using Gradient Descent.

- Training data X is of dimensions $(d \times m)$ where d is the number of features and m is the number of samples. Training labels Y is of dimensions $(1 \times m)$.
- Initialize logistic regression model parameters ω and b where ω is of dimension $(d,1)$ and b is a scalar. ω is initialized to small random values and b is set to zero.
- Calculate Z using X and initial parameter values (w, b)

$$Z = \omega^T X + b$$

- Apply the sigmoid activation to estimate A on Z ,

$$A = \frac{1}{1 + \exp(-Z)}$$

- Calculate the loss $L()$ between predicted probabilities A and ground truth labels Y ,
logistic_loss

$$\text{loss} = \text{logistic_loss}(A, Y)$$

- Calculate gradient dZ (or $\frac{dL}{dZ}$),

$$dZ = (A - Y)$$

- Calculate gradients $\frac{dL}{d\omega}$ represented by $d\omega$, $\frac{dL}{db}$ represented by db

$$d\omega, db = \text{grad_fn}(X, dZ)$$

- Adjust the model parameters using the gradients. Here α is the learning rate

$$\omega := \omega - \alpha \cdot d\omega$$

$$b := b - \alpha \cdot db$$

- i. Loop until the loss converges or for a fixed number of epochs.
2. **Initialize the model parameters.** The weights will be initialized with small random values and bias as 0. While the bias will be scalar, the dimension of the weight vector will be (d X 1), where d is the number of features.
3. Define the **Sigmoid** activation function.

$$\sigma(z) = \frac{1}{1+\exp(-z)} \quad \text{where, } z \text{ is in the input variable.}$$

4. **Logistic Loss Function:** Define the objective function that will be used later for determining the loss between the model prediction and ground truth labels. You need to use vectors A (activation output of the logistic neuron) and Y (ground truth labels) for defining the loss.

$$L(A, Y) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})$$

where m is the number of input data points and is used for averaging the total loss.

5. **Gradient Function:** Define the gradient function for calculating the gradients $(\frac{dL}{d\omega}, \frac{dL}{db})$. Use it during gradient descent. The gradients can be calculated as,

$$d\omega = \frac{1}{m} X(A - Y)^T$$

$$db = \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)})$$

Instead of $(A - Y)$, use dZ (or $\frac{dL}{dZ}$): $dZ = (A - Y)$

Make sure the gradients are of the correct dimensions. Refer to the lecture for more information.

6. Implement the steps for gradient descent mentioned in step 1. Write a function to fit a logistic model with the parameters ω , b to the training data with labels X and Y .
7. Once you have optimal values of model parameters (ω, b) we can determine the accuracy of the model on the test data.

$$Z = \omega^T X + b$$

$$A = \sigma(Z)$$

Submission Directions for Project Deliverables

Learners are expected to work on the project individually. Ideas and concepts may be discussed with peers or other sources can be referenced for assistance, but the submitted work must be entirely your own.

You must complete and submit your work through zyBooks's zyLabs to receive credit for the project:

1. To get started, use the provided Jupyter Notebook in your workspace.
2. All necessary datasets are already loaded into the workspace.
3. Execute your code by clicking the **"Run"** button in top menu bar.
4. When you are ready to submit your completed work, click on **"Submit for grading"** located on the bottom left from the notebook.
5. You will know you have completed the project when feedback appears below the notebook.
6. If needed: to resubmit the project in zyLabs
 - a. Edit your work in the provided workspace.
 - b. Run your code again.
 - c. Click **"Submit for grading"** again at the bottom of the screen.

Your submission score will automatically be populated from zyBooks into your course grade. However, the course team will review submissions after the due date has passed to ensure grades are accurate.

Evaluation

This assignment is auto-graded. There are a total of thirteen (13) test cases and each has points assigned to it. Please review the notebook to see the points assigned for each test case. A percentage score will be passed to Canvas based on your score.